

broken down into 1) Listening to the objection 2) Accepting the objection 3) Seeking & making a commitment 4) Addressing the objection through explicit action.

LAARC: This is similar to LACE objection handling technique with the exception of assess stage where you can ask further probing questions to make sure you understood the objection.

LAIR: This technique is 4 step process as well where you listen, acknowledge, identify the objection. Then trying to reverse the objection by stating the truth.

Feel, felt, found: This technique is based on empathizing with the customer's objection.

Fear, Uncertainty and Doubt (FUD): FUD stands for fear, uncertainty and doubt, and is a method used in sales and marketing to dissuade customers from buying competing products and solutions by providing information that triggers fear and uncertainty, or sows seeds of doubt, about current customer thinking/products. Some of our competitors like to use this technique but we highly recommend leading with facts & not using this technique.

Pushback: This is a technique where you use subtle pushback to object to their objection. Please remember to acknowledge their objection & use empathy while using this technique.

Reframing: This technique suggests reframing the objection so you can turn it to your advantage.

Renaming: This technique suggests renaming the words in your objection so you can turn it to your advantage.

Reprioritize: The technique focuses on being able to handle higher priority objections by reframing their objections & asking other probing questions.

Boomerang: By law of association, if you are stating what they stated to you previously then you have convinced the customer.

Objection Chunking: Taking a higher or lower viewpoint on the stated objection.

Conditional Close: Make closure a condition for resolving their objection.

Curiosity: Ask more questions to clarify the objection.

Deflection: Acknowledge then avoid addressing it directly or mention that you will address it at a later stage. This technique is typically used when you are short on time & the objection has the ability to raise additional questions which can prove detrimental in achieving your objective.

Fallback: This technique suggest that you take a step back without dismissing or deflecting their objection.

Humor: This technique suggests that you diffuse the tension with gentle humor.

Justification: This technique suggests you justify the objection & provide a positive spin to it.

Objection Writing: This is a technique where you use tip the bucket to identify all objections, then writing them down & addressing them one by one to provide a visual aide. This also builds your credibility as a trusted advisor

Pre-empting: This technique suggests you handle the objection before it happens according to your experience.

Objection Handling Role Play Exercise

Presentation

Recording

Facilitating Objection Handling Exercise

Practice Objection Handling in team meeting by using the framework below:

Please note it's suggested to have these meetings weekly, biweekly so handling objections becomes a 2nd nature to us. The

1. Here's a list of objections that we have gathered from the field. This is a starting point & more objections will be added with everyone's contribution. Please feel free to add additional objections to it.

1. Ask for volunteers who would like to participate in the objection handling practice

1. Use a randomized selection (like wheel of misfortune) method to pick an objection & individual(s)(<https://wheelofnames.com/xfb-yz5>) to handle the objections using the techniques above or any way they prefer. Make it fun!!!

1. Please dedicate a fixed amount of time (say 5 mins) to handle the objection with the techniques mentioned above. Remember to pick a technique that feels natural to you.

1. Afterwards, a fixed amount of time (say 10 mins) to have a retro with the team to discuss the objection & brainstorm on different ways to handle the objection. Make sure to take notes in a feedback note like this.

1. Please note the best objection handling method for the particular objection in a summary document like this

1. Repeat the exercise again.

Example Facilitation recording

Best Practices

Before the meeting:

- Gather info about attendees & company (SDR, SAE, LinkedIn, 10k, Job Openings)
- Understand the higher level initiatives if any
- Brainstorm on potential objections, personas, successful meeting outcomes in a strategic meeting with SAE

During the meeting:

- Follow the objection handling process as described above

- Dig deeper with additional questions
- Acknowledge the objection with empathy
- Use objection handling techniques based on your research from the previous step. Feel free to make your own as deemed suitable.
- Try to position the bigger picture where it makes sense without going head-on with the competition
- Parking Lot questions, resources where it makes sense
- Close with definite next steps

After the meeting:

- Follow up within 24/48 hours with resources, questions & next steps

More resources

- SA handbook resource for objection handling & "how to" facilitate additional exercises
- LinkedIn Learning course
- Potential Objections to Ultimate sales plays
- Additional Objection handling techniques for secure & protect customer (SKO 2021)
- 7 Steps to Handling Objections: <https://www.youtube.com/watch?v=DBpLFAS2s3w>
- Changing Minds Objection Handling Techniques
- Solution Architect's O'reilly playlist (objection handling)

SECTION: solutions-architects/sa-practices/gitlab-dedicated-prospects.md

The purpose of this page is to hopefully help an SA onboard when they onboard a new Dedicated Customer, what are the considerations and preparation material during the onboarding/discovery phase. Based on past interactions with existing GitLab Dedicated customers and due to the usually longer nature of onboarding customers, a collaboration project is recommended to keep all stakeholders aware and engaged.

A collaboration template specifically for dedicated customers is available as a template here

Most of the details discussed below are also available as individual issues which provides a good place to include your product managers and customers to collaborate.

Dedicated Reference Links

- GitLab Dedicated Issue Tracker Board
- Customer Onboarding Process
- Slack Channel Dedicated Engineering: fgitlabdedicated
- Slack Channel Dedicated SMEs: sme-dedicated
- Category Direction Page

Administration

- Who will be the administrator to send the GitLab login credentials to?
- What is your preferred time frame for your maintenance window? (4 hour block)
- For emergency security patches and scheduled upgrades, what email address(es) should the notification be sent to?

User Authentication

- Authentication via SAML
- Authentication via LDAP - Please check with dedicated team (Still part of the roadmap)

Connectivity

- To provide VPC Endpoint details
 - IAM Principal for the AWS user or role in their own AWS Organization that will be establishing the VPC endpoint within the Customer account. GitLab Dedicated will use this IAM Principal for access-control: this IAM principal will be the only one able to setup an endpoint to the service.
- List of AZs (up to 2): Which AZs do you want the VPC endpoint to be available in?
- Reverse PrivateLink: What services are required for private link connectivity (E.g. between JIRA, Confluence, Nexus, Artifactory and GitLab)

URL	Service Name	Port
xxx.com	com.amazonaws.vpce.xx-xxx	443

Disaster Recovery/Geo

- Which region would you like your Geo node to be located?
- Do you have specific requirements for which AZ that you want your setup to be located? (Eg. southeast-1a)

Runners

- Where are your runners located?
- Runner Flavors
 - Openshift
- Strategies on how to connect to GitLab Dedicated

- Multi-Cloud
- Leverage on AWS Transit Gateway
- Example Architecture

Domain - Customer Desired Sub-Domains

- E.g.: <CUSTOMERNAME>.gitlab-dedicated.com

Desired Instance Storage & Sizing

- This is inclusive of registry as well as all other resources tied to the application (git, etc)

Logging and Security Requirements

- Are you looking for log streaming to an S3 bucket?
- Log Sources to be streamed to S3 bucket and where?
 - productionjson.log
 - Production.log
 - apijson.log
 - Application.log
 - Applicationjson.log
 - auditjson.log

SECTION: solutions-architects/sa-practices/monthly-release-quiz.md

Solutions Architects need to stay on top of all product changes and with GitLab releasing new features in each monthly update, it can be hard to keep track.

A monthly quiz is organized as a way for team members to test their knowledge on new and existing features.

Participating to the Monthly Release Quiz

The quiz is scheduled every second Thursday of the month. This gives around two weeks for team members to read through the release blog post and get familiar with the new features.

Kahoot is the platform used to hold the quiz. The session lasts 30 minutes with 20 questions. Participants play individually and win points when entering the correct answer as fast as possible.

The top three winners receive a price in the form of a voucher for the SWAG shop. This voucher needs to be spent during a given quarter or during the first month of the following quarter at the latest.

Organizing the Monthly Release Quiz

It is recommended for two people to organize the quiz together.

Step by step

1. Read through the monthly release blog post
2. Each organizer comes up with 10 questions and reviews the other's questions (see Formulating Questions)
3. One organizer enters the questions on the quiz platform, Kahoot. The credentials can be found on 1Password in the shared Solutions Architect folder. If you do not have access to the credentials in this folder, please reach out to Siddharth Mathur or Chloe Cartron.
4. Before the beginning of the quiz, a message is shared on slack to encourage team members to join the event
5. During the event, one organizer shares their screen
6. The event starts by reminding participants on how to join Kahoot. Participants are encouraged to use their real name when playing.
7. Each organizer reads the questions they prepared and provides more information when delivering the answer
8. After the quiz, prizes are requested following this process.
The budget is \$100 per month. The recommended giveaway amount is \$45 for 1st place, \$30 for 2nd place, and \$25 for 3rd place.
9. An email is sent to the winners with information on how to redeem their voucher.

Formulating Quiz Questions

- Quiz questions should be:
 - Fact-based (this means that either docs and/or GitLab release notes are in alignment with the correct answers), rather than opinion-based
 - E.g. "Which security platform is our biggest competitor," this is a bad question because the answer is subjective.
 - Ideally, centered around features and capabilities of GitLab (while there is an emphasis on new features and recent changes, older features are also welcome)
 - True/false, single select, or multiselect format
- When creating questions, the following are good sources to pull from:
 - GitLab documentation
 - GitLab release notes: Overview page, 16.0 release notes example

SECTION: solutions-architects/sa-practices/recognizing-cognitive-bias.md

What is Cognitive Bias:

- Perception that occurs when people are processing and interpreting information in the world around them
- Cognitive biases are often a result of your brain's attempt to simplify information processing
- Our blindspots characterized by unconscious bias
- It affects the decisions and judgments that they make
- Common misconceptions about people: looks, personalities, mental models

Please read this list to understand the 10 most common types of cognitive biases.

How to facilitate Cognitive Bias exercise:

1. Create a survey to collect information & feedback from team members about their own bias.
1. In a small team meeting setting:
 - a. Summarize the results of the survey
 - b. Pick 2-3 examples for a role-playing exercise
 - c. Discuss the results & exercise

Step by Step guide to continuously focus discussions on cognitive bias & psychological safety

1. Pick a topic from this list to discuss during your team meetings at a regular cadence. Plan to dedicate 15 mins to such discussions. The discussion topic can revolve around :
 - Identifying with the bias & to provide some real-life examples
 - What actions can we take if we recognize we have that bias in order to create a psychologically safe environment.
1. Make sure to pick the moderator & a topic for your next meeting.
1. Please use this LinkedIn learning course to schedule deeper discussions around this for the interested audience.

SECTION: solutions-architects/sa-practices/ride-alongs.md

Ride alongs provide Solutions Architects working across segments (SMB, Mid-Market, Enterprise, Public Sector) to participate in calls to learn how Solutions Architects and their customers work through aspects of technical evaluations. The aim is to create an additional opportunity for collaboration, exploration, and growth.

Ride alongs have a Driver and a Passenger. The driver is the one leading the call and is assigned as the primary Solutions Architect for the customer in a given region and segment. The passenger is the Solutions Architect looking to shadow the driver.

Ride alongs can be within a passenger's current segment or region or outside of the given segment or region.

Participating as a passenger in a Ride Along

In order to participate as a passenger in a ride along, a Solutions Architect can leverage the list of drivers on this page to find a Solutions Architect you can accompany.

For passengers interested in the public sector, information within the public sector is less widely available, please reach out to the available public sector drivers to get a ride along coordinated.

1. Review the list of drivers to find someone who has a region or segment of interest to you.
1. Look at the driver's calendar and identify 3 customer meetings of which you would like to ride along.

1. Contact the driver and express an interest in riding along with the 3 meetings. The driver may choose 1-3 of those meetings for you to attend.
1. Sync with the driver before the call.
1. Attend the Ride Along through a calendar invitation in the call.
 1. Introduce yourself in the call.
 1. Assist the driver throughout the customer call by contributing to any Running Notes which should be found in the Customers & Prospects folder.
1. Debrief after the call. Use the Feedback Description to populate your Troops Activity upon completion.

List of Ride Along Drivers

Any Solutions Architect is eligible to be a driver. If you opt-in, please ensure your calendar is available for others to view

To become a driver, submit an MR to this page adding yourself to the table below and assign your manager as a reviewer.

Team Member	Segment	
----- -----		
Tim Poffenbarger	Americas SMB, Mid-Market	
Noah Ing	Americas SMB, Mid-Market	
Jeremy Wagner	Americas SMB, Mid-Market	
Madou Coulibaly	EMEA, Enterprise	
Alexander Dess	EMEA, Enterprise	
Dominique Top	EMEA, Enterprise	
Sameer Kamani	PubSec	
Linc Williams	PubSec USN, USMC,DHA	
Ben Ridley	APAC PubSec	
Sophia Manicor	Americas SMB, Mid-Market	
Ken McKnight	Americas Enterprise West	
Jefferson Jones	Americas SMB, Mid-Market	
Edmond Chan	Americas Enterprise East.	

Sync before Call

Passengers before a customer call should:

- Review the SA Triage Issues Checklist, recent running notes, or Troops activities related to the account or opportunity
- Introduce themselves to Account Executive
- Discuss findings in reviewed Chorus Calls
- Align on messaging to the client and discuss desired outcomes of the call

Introducing yourself

Passengers should use the ride-alongs as an opportunity to practise their introductions as trusted advisors. Passengers should focus on being friendly and open. If possible, mention a personal technical experience that resonates with the audience (for example: if a customer

plans a migration from Jenkins to GitLab CI and you have already done that yourself in the past).

For example

Hello, my name is \${PASSENGER}, and I am working along side \${DRIVERNAME} to:

- Listen to your technical and business needs
- Act as your technical resource
- Help guide you through evaluating GitLab

It's a pleasure to meet all of you today.

Debrief post Call

Passengers debriefing after a customer call should:

- Review any action items to follow up on
- Discuss if messaging to client was successful completed
- Provide feedback to the driver on call
- Mention your shadow experience in solutions-architects to help promote ride alongs!

Rattle Feedback Template

Record your ride along activity in Rattle as a call to the account (and not the opportunity) with the appropriate activity type. Recording at the account level is required because ride alongs that are inter-segment or inter-region will not have their opportunities available to the riders.

The following can be used to guide how you record the description of your troops activity.

shell

Call Duration:

Region/Segment Currently Assigned:

Region/Segment Ride Along was in:

Differences in how the call was conducted:

Differences in how the customer interacted:

Key Takeaways that you'll be incorporating into your region/segment

SECTION: solutions-architects/sa-practices/sa-office-hours.md

Office Hours are a collaborative space where AE/CSM/SAs can learn and ask questions to the Commercial Solutions Architecture team.

Participating in Commercial SA Office Hours

Office hours are in 25 minute blocks and will occur bi-weekly.

Preparing for Office Hours

- Please be logged into your GitLab Zoom account.
- The meeting will be canceled if there are no questions.
- For Hosting SAs: Ensure that responses are timeboxed (4 minutes or less). SA 3 on rotation will be the notetaker and timekeeper.

SECTION: solutions-architects/sa-practices/sa-retrospective-feedback.md

The worldwide Solution Architecture team, as a learning organization believes that we can be more successful by receiving feedback. Feedback is a two-way street. It is important that SA management takes the time to provide feedback on a regular basis to their individual team members, but simultaneously GitLab team members should share their feedback with management and peers. During this two-way feedback exchange, it provides SAs insights into their activities working with customers and internal processes as well as the SA management insights into their leadership skills.

Solution Architecture Retrospective Feedback

GitLab as an organization being fully remote, encourages Solutions Architecture groups and other teams to run agile retrospective feedback sessions. As an organization operating in various worldwide regions, we are distributed and coming together in-person in a meeting room may not always be possible. In the spirit of one of our company values Iteration it gives us an opportunity to hold agile retrospective meetings that follow similar software development principles in the context of our engagements to educate and enable our customers. During the retrospective, the team reflects on what happened in the iteration and focussed on actions to be taken to improve going forward.

On a high-level goal, we are trying to understand what happened in a given time period (on a quarterly basis for instance), with a goal to improve "things" in the upcoming time period which is strictly based on those learning and discussions.

We suggest for teams to pause and reflect on what transpired and encourage collaboration to distinguish between what worked from what didn't work during a particular time period. It also allows for Solution Architecture teams to re-align given field observations coming from our customers.

The following diagram shows the Solution Architecture Retrospective Overview.

```
<div style="width: 640px; height: 480px; margin: 10px; position: relative;"><iframe
allowfullscreen frameborder="0" style="width:640px; height:480px"
src="https://lucid.app/documents/embedded/da94d876-9df6-4916-b5f4-a48d0f5fea99"
id="eSy6JdDjATk"></iframe></div>
```

Solution Architecture Retrospective feedback critically impacts customers and other cross-functional teams, including Product, Sales and Marketing. Based on the retrospection insights, Solution Architects can improve product alignment and prioritisation. Apart from the Solution Architecture team, many other cross-functional teams will benefit from Solution Architecture Retrospection Feedback and use it to improve internal alignment and enhance the customer-centric approach.

What is an agile retrospective feedback session?

Agile feedback retrospective sessions can end up in pointing out the flaws and challenges which teams encounter. However, it is important that team members also equally consider to bring up positive experiences and iterate on improvements which can increase productivity and efficiency in their roles.

Retrospective feedback sessions should ideally consider 3 areas:

1. "I'm glad that..." >> Green
1. "I'm wondering about..." >> Yellow
1. "It wasn't so great that..." >> Red

The Solution Architecture team utilizes Postfacto as the main technology for agile retrospective feedback sessions. This is a digital whiteboard tool. The GitLab SA team has configured and implemented it internally. We run private retros which generally are password protected for teams. This Postfacto internal session is hosted within our internal environment and are not publically accessible. Since it is an internally hosted solution, it is available to all internal GitLab teams and not only restricted to the SA team.

Access instructions for our internal Postfacto instance

How can I host a retrospective feedback session with Postfacto

The summary steps are as follows:

1. Use the Postfacto account to log in to the Postfacto panel.
1. Create a retro for the retrospective feedback session you plan to host.
1. Share the retro URL and the password with the participants before the retrospective feedback session.

For details, please refer to this document (Internal members only).

A walk-through of setting up a new retrospective can be found in the video below.

<iframe width="560" height="315" src="https://www.youtube.com/embed/NhZRkpmGVsk" title="YouTube video player" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share" allowfullscreen></iframe>

What are the roles during an agile retrospective feedback session?

Agile retros are commonly used as a practise withing engineering teams. However, the GitLab SA groups has implemented a way to reflect on how we work in order to become better in what we do. One of our approaches is the benefit of hosting retrospectives. We hold retrospective sessions towards the end of our fiscal quarter to seek feedback (but that is not restricted to that period only). In the context of our group, the Solutions Architecture leader (i.e. SA Manager, SA Director, SA VP etc.) takes the lead as the host of the restrospective workshop by creating a safe space to gather peoples' opinions, fascilitating discussion and identifying what the team members need.

The entire group in the retrospective workshop becomes a participant with the opportunity to collaborate, share insights and have a voice.

The host of the restrospective feedback session is encouraged to share the screen over zoom or when in-person, project the retro board for everyone.

Recommended guidance for all retro participants:

- Become an effective listener during the session
- be open-minded and consider every participants experience and opinion
- A retro feedback is not meant to be taken personally
- Focus on actions to drive improvements within the team
- A retro session does not need to run for more than 60 mins

How do I participate in an agile retrospective feedback session?

We encourage that SA Leaders execute retros on a regular basis proactively. However, it is also encourages that individual team members help monitor the frequency of retros and also proactively engage by requesting sessions. As the team leader, we are responsible to be inclusive to invite the entire team making everyone a participant. Optionally the team leader can include relevant team members from immediate teams/groups to participate.

The following activities are required to hold a retrospective session:

1. A Postfacto session with an URL and password (assigned during retro configuration)
1. A calendar invite with the access to the retro session
1. Optionally: a meeting notes document to record actions from the retro

It is recommended to create a new retro board ahead of time by setting a name and chosing an URL. This will allow participants to start providing feedback in the Postfacto tool ahead of time by adding retro items to each column for discussion. During that time team members are also encouraged to start voting if they feel strongly about already raised insights/suggestions (by clicking on the heart icon).

How do I start and end a retro session?

Given the nature of retros are considering opportunities to celebrate improvements ("Happy"), or trying to obtain more clarity around specific topics ("Wondering") or suggest opportunities for improvement ("Sad?"), those are the discussion columns within the retro board. It is suggested to start by picking a first retro item randomly but considering the number of votes of hearts. The retro host is keeping an eye on the retro time-frame which has a 5-minute timer in the retro item (can be configured).

The guide for the retro item discussion also requires to add actions to improve teamwork. The team needs to nominate a team member to capture actions in either the Postfacto retro tool (at the bottom of the UI) or in a separate team meeting notes document.

The community around the open-source Postfacto technology and its partners/collaborators have also built this blog "How to Run a Really Good Retrospective" for further guidance.

Important note: The retro time-frame should not be more than 5-minutes per item, as a result of that and for an effective discussion, the participants do not need to go into extensive descriptions "why" that particular item is critical nor into "trying to identify" a solution. The idea for the discussion is to quickly reflect and suggest a best next step or an action. The action then will become a means to identify a solution.

As the agile retro host, close the retro by reflecting on the session and thanking the team for their candid and honest feedback. Focus on great accomplishments of the team and the great interaction during the session.

Follow-up and challenge takeaways

There are a few ways to take the action items from the retro and integrate them into the ways of working depending on the scope of the actions.

For small items changes can be made immediately, updating documents and handbook pages.

When looking at larger changes to processes the best course of action is to integrate these action items into cross-functional OKRs. Especially in these cases with cross-functional process changes it is important to socialise these changes in the appropriate slack channels as well.

SECTION: solutions-architects/sa-practices/selling-professional-services.md

This is a practice guide for a Solutions Architect (SA) to position Professional Services to a customer during the sales cycle and gather the appropriate information to provide to the Professional Services Engagement Management Team for custom scoping, if necessary. The overall workflow for selling Professional Services is defined in the Professional Services Handbook - Selling Professional Services page.

- See the Positioning Professional Services Class on LevelUp for an overview.

Types of Services

GitLab provides a wide array of Professional Services offerings to accelerate the customer's time to value in GitLab platform adoption and maturing DevOps practices. Service offerings range from standard instructor-led training to custom services packages that address a specific customer need. Generally, Professional Services are sold in one of two ways:

- via a sku on an order form - no SOW is needed for the customer to purchase these services; they can be added directly to an order form, in conjunction with a license purchase, when applicable. the Professional Services Catalog has a full list of sku-based services offerings. The most popular sku-based services are:

- GitLab Rapid Results - self-managed for implementing GitLab in a cloud environment
- GitLab Migration+ for migrating up to 500 projects and 1000 repositories from another git-based system to GitLab (SCM only - CI//CD is not included)

- GitLab Expert Services - a block of 40 hours to help a customer with whatever adoption needs they have

- via an SOW as a "GitLab Service Package" - this is used for any custom package of services that are defined to address a specific customer need. A custom SOW is typically used for these purposes:

- bundling together source code migration and GitLab CI adoption/transition from other CI tools such as Jenkins

- complex self-managed GitLab implementations within the customer's data center

- health check services for existing customers, where the details of the offering and required effort will depend on the specific customer circumstances

- migrations to GitLab where the Migration+ offering is not an appropriate fit

Cross-team Collaboration

It is expected that the Solutions Architect lead in the positioning of Professional Services for new license opportunities as they transition from stage 3- Technical Evaluation to stage 4- Proposal. Solutions Architects should be able to articulate the value of using Professional Services vs the customer doing the work on their own and should be able to choose the appropriate Professional Services offerings that will most benefit the customer. If it is determined that a custom service package is likely the best option, then the Solution Architect or appropriate sales role (AE, SAE) should initiate the custom services scoping workflow by using the services calculator to create a scoping issue. A Professional Services Engagement Manager will pick up the scoping issue and will collaborate with the account team to scope the appropriate services package.

When an existing customer is interested in Professional Services and there is not a current license opportunity, the CSM would typically work with the AE/SAE to identify the appropriate services and custom scope any needs, when applicable. The CSM and/or PS Engagement Manager may request SA assistance when the customer has specific questions regarding GitLab capabilities as they are considering Professional Services options, e.g. if a customer is considering migrating from self-managed to SaaS, they may have questions about the differences between self-managed GitLab features vs. using the SaaS platform, limitations of SaaS, SaaS security considerations, consumption fees, etc.

We want to keep the customer experience in mind when people in multiple roles are interacting with the customer. To this extent, it's important that:

- we share information we've learned about the customer internally in preparation for any call

with them - while we want to validate and extend the depth of understanding of a customer's situation, we should not be repeatedly asking them the same basic questions.

- we act as a unified team helping the customer to reach their desired goals, documenting open questions, and following up with the appropriate team members to ensure that customer questions are being addressed.

As a guideline, team member responsibilities will break out as follows:

| Role | Activities |

| --- | --- |

| Account exec (AE/SAE) | Manage overall customer relationship, oversee and negotiate any contract terms with the customer, including subscription and services agreement, manage salesforce opportunity through closure |

| Solutions Architect | Answer technical questions related to the GitLab platform and unrelated to Professional Services offerings and approach, position PS options, and perform initial customer discovery for customers/prospects with active license opportunities|

| Customer Success Manager | Position PS options and perform initial customer discovery for customers without active license opportunities |

| PS Engagement Manager | Elaborate on details of PS approach and different services options, answer questions about customer expectations for PS engagement and prerequisites and preparation, determine the final scope of work and generate an SOW, shepherd SOW through an approval process, transition services engagements to the delivery team |

Discovery for Custom Services Scoping

To appropriately scope custom services engagements, the PS Engagement Manager will need details about the customer's current tool usage and the goals of GitLab. Having a reasonable understanding of the customer needs ahead of a PS scoping call will help the PS Engagement Manager be efficient and productive in arriving at an appropriate services package that suits the customer's needs. The PS Engagement Manager will provide guidance on what information is needed based on the specific scenario, but the list below is a reasonable starting point for most custom PS:

- Number of users currently using GitLab (if applicable) and number of planned users when GitLab has been fully rolled out
- Customer DevOps goals and priorities in adopting GitLab
- What tools is the customer transitioning from and replacing with GitLab? SCM (GitHub, Bitbucket, Svn, Perforce, etc), CI/CD (Harness, Jenkins,...), Project Management (Jira,...), Container and/or package registries, other?
- What tools will the customer plan to integrate GitLab with?
- What is the customer's planned deployment architecture? SaaS or self-managed? If self-managed, single node omnibus or HA implementation, and in a cloud (which one) or in the customer's data center?
- If looking to adopt GitLab CI/CD, what types of applications does the customer have? Are they deploying to a cloud environment? which one(s)? Do they deploy apps in containers? Are they using k8s for container orchestration?
- What version of GitLab is the customer on, or planning on deploying? And what license tier - Premium or Ultimate?
- If the customer is looking to migrate from GitLab self-managed to SaaS, it will be helpful to have the customer run the Evaluate script. This tool will provide a count of users and

repositories, show which GitLab features are used in each project, and flag any project where the type or amount of data might be problematic for migration.

The Professional Services Team keeps a more detailed list of scoping questions pertaining to different types of services that customers request. Details can be found on the Professional Services EM Scoping Guidelines page.

SECTION: solutions-architects/sa-practices/workshops.md

Common workshop topics include:

- DevOps Automation
- Portfolio and Project Management
- Security
- Advanced CI/CD

These sessions accelerate customer comfort and awareness while helping to refine and reduce technical evaluation requirements. Workshops often minimize or eliminate the need for extensive POVs and can lead directly to technical wins.

Technical Requirements

Each workshop requires:

- A dedicated GitLab instance for attendees to use during exercises
- Pre-configured class projects
- Setup of issue tracking workflows
- Additional environment preparation specific to the workshop topic

For more information, review the Hands-On Workshop Tools and Resources handbook page.

Capacity and Format

To ensure optimal experience:

- Attendance is limited to 200 or fewer participants
- This limit enables:
 - Stable working environments for all attendees
 - Timely responses to questions and issues
 - Effective hands-on guidance

Key Benefits

The workshop approach delivers multiple strategic advantages in the pre-sales process:

- Dramatically accelerates customer comfort and awareness with GitLab's extensive feature set through direct, hands-on experience
- Substantially reduces or completely eliminates the requirement for time-consuming and

resource-intensive POVs

- Provides technical teams with immediate, practical experience in applying GitLab to their specific use cases
- Often catalyzes technical wins by demonstrating clear value alignment with customer needs
- Creates an interactive environment for deep engagement with technical stakeholders
- Builds confidence in GitLab's capabilities through guided, hands-on exploration

Audience

Workshops are strategically designed to engage a diverse technical audience, typically including:

- Software Developers and Engineering Teams: Those directly responsible for code development and deployment
- DevOps Practitioners: Professionals focused on streamlining development and operational processes
- Security Professionals: Team members responsible for application and infrastructure security
- Operations Team Members: Staff managing infrastructure and deployment environments
- Quality Assurance Engineers: Those responsible for testing and quality processes
- Platform Engineers: Professionals designing and maintaining development platforms
- Technical Architects: Those making technical decisions about tool adoption and integration

Workshop Format

Structure

1. Customer Background Discussion and Context Setting

- In-depth exploration of the customer's current technical landscape
- Detailed discussion of challenges and pain points driving GitLab evaluation
- Establishment of specific workshop objectives and success criteria
- Understanding of customer's existing toolchain and integration requirements
- Review of current development processes and desired improvements

2. Comprehensive Demonstration Phase

- Strategic showcase of GitLab capabilities aligned with customer needs
- Live demonstration of relevant workflows and integrations
- Real-world examples of how GitLab addresses specific customer challenges
- Illustration of best practices and recommended approaches
- Interactive Q&A to address specific technical questions

3. Guided Hands-on Portion

- Step-by-step walkthrough of key GitLab features and functionality
- Practical exercises designed to reinforce learning objectives
- Direct assistance with configuration and setup
- Collaborative problem-solving sessions
- Real-time feedback and guidance from GitLab experts

4. Independent Hands-on Labs

- Self-directed exploration of GitLab features
- Application of learned concepts to customer-specific scenarios
- Opportunity for deep-dive into areas of particular interest
- Practice with advanced features and integrations
- Freedom to experiment with different configurations and approaches

Delivery Options

The workshop format offers flexible delivery options to accommodate various customer needs:

In-Person Sessions:

- Direct, face-to-face interaction with GitLab experts
- Immediate assistance with technical questions
- Enhanced collaboration opportunities
- Stronger relationship building
- Easier handling of complex technical discussions

Remote Delivery via Zoom:

- Broader accessibility for geographically dispersed teams
- Recording capabilities for future reference
- Screen sharing for detailed technical demonstrations
- Breakout rooms for focused discussions
- Chat functionality for resource sharing and quick questions

Duration and Timing:

- Typical length: 2-4 hours, allowing for comprehensive coverage
- Strategic breaks to maintain engagement
- Flexible scheduling to accommodate different time zones
- Optional follow-up sessions for advanced topics
- Built-in buffer time for extended discussions

Workshops vs Demonstrations

Understanding the distinct characteristics of workshops and demonstrations is crucial for effective pre-sales engagement:

Aspect	Workshop	Demonstration
-----	-----	-----
Duration	Extended engagement (2-4 hours)	Focused presentation (30-60 minutes)
Scope	Comprehensive coverage of multiple use cases and features	Targeted exploration of specific functionality
Interaction Level	High engagement with hands-on activities, exercises, and practical application	Primarily observational with Q&A opportunities
Problem Focus	Broad exploration of use cases with deep-dive capabilities	Specific solution demonstration for identified challenges

Audience Engagement	Active participation required, with practical exercises and independent exploration	Passive viewing with occasional interaction through questions
Resource Investment	Significant preparation and delivery time, requires dedicated technical resources	Minimal preparation, can be delivered by individual team member
Outcome	Comprehensive understanding and practical experience	Basic feature awareness and solution visibility
Follow-up Requirements	Often reduces need for additional technical evaluation	May lead to requests for more detailed exploration

Workshop Preparation Best Practices

Essential Preparation Steps

1. Comprehensive Dry Runs

- Execute multiple complete rehearsals of the workshop content
- Practice handling various technical scenarios and potential issues
- Prepare detailed verbal explanations for complex concepts
- Anticipate and prepare for common customer questions
- Document potential technical challenges and their solutions
- Test all demonstrations in multiple environments
- Verify all links, codes, and access requirements
- Practice recovery procedures for potential technical issues

2. Interactive Engagement Strategies

- Implement regular checkpoints for participant questions
- Create natural breaks in content for discussion
- Prepare discovery questions to gather valuable customer insight
- Design interactive elements to maintain engagement
- Plan for different learning styles and technical levels
- Create opportunities for peer-to-peer learning
- Develop contingency plans for low participation

3. Resource Management and Organization

- Prepare comprehensive resource documentation
- Create easily accessible link repositories
- Organize session registration processes
- Document all required project materials
- Prepare backup copies of essential resources
- Create clear instruction sets for all activities
- Maintain updated reference materials

4. Structured Delivery Framework

- Develop clear, step-by-step instruction sets
- Create visual aids for complex concepts
- Implement consistent progress tracking methods
- Follow established presentation best practices

- Maintain clear communication protocols
- Create detailed session timelines
- Prepare transition strategies between sections

Companion Role and Responsibilities

A dedicated workshop companion serves as a crucial support resource, responsible for:

Technical Support:

- Maintaining comprehensive knowledge of workshop content
- Managing Learn Labs session access and technical issues
- Providing real-time troubleshooting assistance
- Monitoring participant progress and identifying struggles
- Managing technical infrastructure during the session

Communication Support:

- Managing question queues and discussion threads
- Sharing resources and links in chat
- Documenting key discussion points
- Capturing follow-up items and action points
- Facilitating breakout sessions when needed

Documentation:

- Maintaining detailed session notes
- Recording key technical questions and answers
- Documenting customer-specific requirements
- Capturing feedback and improvement suggestions
- Creating summary reports post-workshop

Backup Coverage:

- Being prepared to take over presentation if needed
- Managing technical difficulties
- Handling unexpected situations
- Providing additional examples or explanations
- Supporting individual participants as needed

Success Metrics and Evaluation

Workshops can be evaluated based on several key success indicators:

Immediate Outcomes:

- Reduction or elimination of POV requirements
- Achievement of technical acceptance
- Positive participant feedback
- Increased understanding of GitLab capabilities

- Clarification of technical requirements

Long-term Impact:

- Acceleration of sales cycle
- Increased GitLab adoption rates
- Improved customer satisfaction
- Reduced implementation time
- Enhanced customer relationship development

Measurement Methods:

- Participant surveys and feedback forms
- Technical requirement resolution tracking
- Sales cycle duration analysis
- Implementation success metrics
- Customer satisfaction assessments

SECTION: solutions-architects/sa-practices/communities-of-practice/_index.md

This is a collection of best practices collected from working with customers on each stage of the Software Development Lifecycle (SDLC).

This list is not exhaustive but provides the Solution Architect (SA) with a few good pointers when preparing for a meeting with a prospect asking about industry best practices surrounding a topic or stage.

Stages

Stage	Topics
Manage	N/A
Plan	N/A
Create	N/A
Verify	N/A
Package	N/A
Secure	N/A
Release	N/A
Configure	N/A
Monitor	N/A
Govern	N/A
Integration Technologies	1. Elasticsearch: Advanced Searching

Contribution Guidelines

How can I contribute? What should be included in the topic?

The write-up should not be lengthy and emphasises the value of why this best practice should be

adopted by other organizations.

An topic to highlight a certain value should include the following:

markdown

[TEMPLATE]

1. Why is this valuable to a customer?
2. What are the details of the best practice? (E.g. how to setup, how to configure)
 - a. Do not replicate existing artefacts from the documentation but provide links on where to best find them
3. Any demo videos or links that might help an SA prepare for a meeting more efficiently
4. Typical Q&As that customers tend to ask around this topic?
5. Other links for references [OPTIONAL]

SECTION: solutions-architects/sa-practices/communities-of-practice/integration-technologies.md

This is a collection of best practices collected from working with customers on each stage of the SDLC. This list is not supposed to be exhaustive but provide the SA with a few good pointers when going into a meeting on a specific stage or feature.

Topics List

| Topics | Description |

| ---- | ----- |

| 1. Elasticsearch - Advanced Search | Enabling Elasticsearch for faster and more accurate searching of artefacts in GitLab |

Advanced Search with Elasticsearch

Search is a very important aspect in our everyday lives. From using navigation apps to guide us to our restaurants nearby to translating words that are not in our native language, it is crucial that the search functionality is fast, accurate and flexible to allow us to get the most relevant results possible.

With integrations with Elasticsearch, we are able to leverage on the Lucene library to provide advanced search functionalities for GitLab users.

Architecture

Elasticsearch leverages on clustering to search, distribute tasks and index across nodes to achieve high performance search. Unless deployed in a test environment, typically an Elasticsearch cluster should have at least 3 nodes to establish quorum.

As the cluster grows, more nodes can be added to the cluster to improve concurrency of users or improve resiliency

Installation & Integration

The global search team maintains a page on the basic "know-hows" of setting up Elasticsearch and integrating with GitLab.

Installation Options

Elasticsearch is able to install on most platforms. Inline with our recommendations to push customers to adopt Kubernetes, Elasticsearch also has an official Kubernetes operator and docker image that simplifies the deployment process and helps the cluster to scale out quickly as determined by search traffic.

Demo Resources & Examples

Demo Examples:

- Recently Viewed Suggestions [VIDEO]
- Advanced Search Syntax

FAQ

Question: Do you recommend running Elasticsearch on the same host as GitLab?

Answer: No. Elasticsearch will consume memory and file descriptors that is not allocated to the JVM heap (e.g. results caching, aggregations, etc.) As a result, it may result in resource contention of the various systems resulting in instability of the whole setup.

Question: Are Elasticsearch operations transactional?

Answer: No, not out of the box. Elasticsearch was not designed to be ACID compliant.

SECTION: solutions-architects/sa-practices/customer-success-plan/_index.md

What is the Mutual Customer Success Plan

The Mutual Customer Success Plan Template is a merging of the Pre Sales Customer Success Plan and the Post Sales CSM Customer Success Plan template. It contains the following:

- Customer Background
 - Customer Business Outcomes
 - Key Business stakeholders
 - Current and Desired State / Workflows
 - Technical Activities Planned or Completed
 - Technical Evaluation Plan List
- Customer Phased Success plan & Adoption Journey

- Proposed Initiatives and Milestones

Please use the Mutual Customer Success Plan Template for all Strategic and Enterprise opportunities.

Please use the Commercial Customer Success Plan Template for all High Velocity or Commercial opportunities.

Why Use a Mutual Customer Success Plan (CSP)

Primary Objectives

A Mutual Customer Success Plan aims:

1. For the Customer: To be prescriptive on a phased successful adoption of GitLab, including architecture recommendations, deployment, migration, integration, onboarding users and applications at scale, along with recommendations for any automation processes as well as dashboards and monitoring. As an SA you should always collaborate with the customer, to create the Mutual Success plan and help them understand, preliminary, what is involved in adopting GitLab as their AI powered DevSecOps Unified Platform, from not only a technological or deployment point of view, but from a people and process point of view.

1. For the Customer Success Manager (CSM): Collaborate with CSMs for all deals over the appropriate threshold (currently 300k Net ARR), as it transitions from pre to post sales. It is important to provide the CSMs with all the relevant context as they take over the relationship. As an SA, you collaborate with the CSM to help them understand the customer's objectives and milestones for the next 30, 60, 90 days as well as for the next 6 months. The CSM can then formulate a cadence and enablement plan. The CSM will be responsible to update and maintain the Mutual Customer Success Plan after the sale. It will serve as a customer facing document, helping to facilitate the customer achieving their initiatives, and objectives. A CSP should be created and reviewed w/ the customer even in cases where there are no CSMs assigned.

1. For the Professional Services Team: Present the Customer Success Plan with the scope of initiatives that can be used to determine a Statement of Work.

1. For the Go To Market Team: When a GitLab executive sponsor or anyone in the GTM or customer support needs to be engaged, the Mutual CSP acts as a source of truth to provide context between the customer's and GitLab's interactions.

When to Create a Mutual Customer Success Plan

A Mutual Customer Success Plan should be initiated during stage-2 (scoping) and the first version should exist before going into stage-3 (technical evaluation). This is when an SA begins to prescribe the GitLab solution to the prospect or customer's objectives and what a phased adoption and customer adoption journey would look like, in collaboration with the customer.

The Mutual Customer Success Plan should be reviewed and accepted with a customer at some point before stage-4.

Opportunity Thresholds for Creating and Reviewing Mutual Customer Success Plans?

1. All New or Growth Opportunities with \$300k net ARR or more, in stage-3 (technical evaluation) and beyond. (The SA is the DRI, which is handed over to the CSM)
1. All opportunities with \$30K net ARR or more for focus accounts as indicated in salesforce. (Opportunities that are part of a focus account have salesforce named "Compensation Target Account" checked.)
1. All opportunities where the SA is involved in a SA Validated Tech Evaluation.

SA Managers have discretion for additional opportunities to be included. Additionally, SA Directors can provide an exception in specific cases where it is impossible or unreasonable for the SA to create a CSP for a customer.

Once an opportunity has been identified using the criteria above, as requiring a Customer Success Plan, the Solutions Architect should initiate it at the end of Stage-3 Technical Evaluation, and place the URL in the Customer Success Plan field, in the opportunity, in Sales Force.

Then the SA ensures that the relevant fields in the Customer Success Plan section, on the Opportunity level in SFDC, is maintained throughout the process.

CSP Exception Process

When it is unreasonable or impossible for the SA to create a CSP for a customer, the SA Director can mark the opportunity as requiring a CSP exception. This provides us a feedback driven approach to iterating on our customer engagement models, encourages dialogue on specific opportunity feasibility, and provide a KPI based approach to SA customer success planning that is within the control of the SA organization.

Possible reason it would be unreasonable to create a CSP.

- The SA is unable to engage with the customer and we've made multiple attempts with the customer.
- The customer is not interested in engaging on a CSP after we've explained the necessity and benefits.
- Sales leadership (VP+) supports not doing the CSP based on the dynamics of the opportunity.

The SA and SA leadership should work with the sales team to figure out how to engage on a CSP before marking it for exception

To mark an opportunity with a CSP Exception, following the guidance for Tracking customer success plan creation and review below.

Tracking Customer Success Plan creation and review

To drive visibility, accountability, and collaboration, CSPs and the associated customer review need to be recorded on the associated opportunity in salesforce. Please use the following fields:

1. Customer Success Plan: field for the URL to the plan. If CSM Exception is checked, this

field will contain the justification for the exception (using this field is a temporary method while a field dedicated for CSM exception justification is implemented).

1. CSM Exception: SA Director+ can check this if it's impossible or unreasonable for the SA to create a CSP for the customer.

1. Customer Accepted: Check if the CSP has been reviewed with the customer. The customer doesn't need to formally "accept" the CSP for this to be checked.

At a High Level What is the Customer Success Plan Process

The Customer Success Plan process is outlined here.

Please pay attention to the responsibility of the SA to update the opportunity in Sales Force with the Customer Success Plan URL as appropriate.

How To Build a Customer Success Plan

Let's describe in more detail the Day on the Life of an SA.

Every meeting that is conducted with the customer, is an opportunity for you to understand the customer's challenges and outcomes desired.

The following is a recommendation on how to advance an opportunity, meeting by meeting, stage by stage, and what information you should gather to first build the Customer Success Plan.

1. Customer Success Plan Draft (Stage-1:Discovery to Stage-3:Technical Validation)

- Meeting 1 -Discovery - Discover the Desired ~~~Challenge~~ Outcome [Why Change?]

- During the Meeting

- Research and confirm "1. Company Background"

- Understand who are the "3. Key Stakeholders":

- 1. Executive Decision Maker / Economic Buyer,

- 2. Technical Champion/Coach / Buyer,

- 3. Process or Operational Owner, and get introductions

- Ask about the "4. Business Outcomes / Strategic Initiatives"

- Start Discovery on any Tool Chain per Stage / Processes etc.

- Whiteboard or Present very high level GitLab value

- After the Meeting

- Create a Customer Success Plan with the following content from your first Meeting

- "1. Company Background"

- "2. What We have Heard"

- "3. Key Stakeholders"

- "4. Business Outcomes / Strategic Initiatives"

- VERY IMPORTANT: Update the Customer Success Plan URL in the opportunity in Sales Force.

- If you don't have any of the above information, because most of the time was spent answering the customer's technical questions, schedule another discovery meeting to gather the info.

- Next Steps:

- Current and Desired Workflow Discussion

- Can also do a brief "Drive by" Demo, which is a short 5 minute demo to facilitate more discovery

- Meeting 2 -Scope - Scope the Workflows to Achieve Desired Outcomes [Why Now?]

- During the Meeting
 - Present "2. What We have Heard" so far
 - Continue any Discovery on any Tool Chain per Stage / Processes etc. that would form the foundation of a "5. Current and Proposed Workflow Diagrams"
 - Ask about all "6. Applications & Use Cases / Risks & Mitigations" if all desired Use Cases are not accomplished.
 - Present or Whiteboard GitLab Use cases and Value
- After Meeting
 - Update the Customer Success Plan with
 - "2. What We have Heard"
 - "5. Current and Proposed Workflow Diagrams"
 - "6. Applications & Use Cases / Risks & Mitigations"
 - Start a "8. Technical Evaluation Plan & Activities" on what will be done to achieve a Technical Win and Decision.
 - Next Steps:
 - Present Current and Desired Workflow Diagrams
 - Optional (Propose Workshop or Demo).
- Meeting 3 -Scope - Qualify the Value [Why GitLab?]
 - Meeting Prep
 - For Meeting Prep, work with your Account Executive, to shore up "2. What We have Heard" so far.
 - Start to develop your "8. Technical Evaluation Plan & Activities", that is, what exactly will be done to achieve a Technical Win and Decision.
 - During the Meeting
 - During the third meeting Present an updated "2. What We have Heard" so far
 - Present the "5. Current and Proposed Workflow Diagrams" and solicit feedback and discussion.
 - Present the "6. Applications & Use Cases / Risks & Mitigations" and solicit feedback and discussion.
 - Make sure the customer understands our value in meeting their outcomes and processes.
 - Determine how a Technical Decision would be made and document in "8. Technical Evaluation Plan & Activities". Get agreement from Customer on the path to making a Technical Decision.
 - Next Steps:
 - SA and AE to propose the next steps for Business and Technical Validation: Workshop / Demo / POV / Value Stream Workshop / Business Value Assessment
- Meeting 4 - Business & Technical Validation Activities - Validate the Business and Technical Value
 - Meeting Prep
 - For Meeting Prep, work with your Account Executive, to review your "Customer Success Plan Activities" on what will be done to achieve a Technical Win and Decision.
 - During the Technical Evaluation
 - Conduct your Technical Evaluation Plan and Activities as was decided between GitLab and the customer, e.g. Workshop, POV, Demo, Reverse Demo, Day in the Life
 - Work towards the Technical win (may require more than 1 meeting)
 - During and After the Technical Win
 - Update "Activities Planned / Completed"
 - After the Technical Win, create a "Value Outcome Map / Metrics".

- The Outcome Map traces the Customer's Outcomes to be achieved, with their Strategic Initiatives, mapped to GitLab's capability and what value will it bring. This would be used in the proposal.

- Next Steps:

- After securing Technical Win, work with AE on a Proposal
- Initiate the Customer Success Plan (see below)

1. Mutual Customer Success Plan (Stage-3:Completed)

- Once Technical Validation is completed, update the Mutual Customer Success plan
- "Company Background"
- "What We have Heard"
- "Key Stakeholders"
- "Business Outcomes / Strategic Initiatives"
- "Current and Proposed Workflow Diagrams"
- "Applications & Use Cases / Risks & Mitigations"
- "Activities Planned / Completed"
- "Value Outcome Map / Metrics"

1. Meeting 5 -Proposal - (Stage-4:Proposal and beyond) - Proposal and Recommendations to Achieve Customer's Outcome

- Prep with your AE on a Proposal / Champions Deck that includes the following
- "What We have Heard"
- "Business Outcomes / Strategic Initiatives"
- "Current and Proposed Workflow Diagrams"
- "Value Outcome Map / Metrics"
- After Proposal Presentation, continue to update and complete the Mutual Customer Success plan
- VERY IMPORTANT: Ensure the Customer Success Plan URL in the opportunity in Sales Force is updated.
- Next Steps:
 - While Negotiation occurs, As the SA, set up meetings with the customer and with the CSM to add the following to the Mutual Customer Success Plan:
 - "Users / What was bought / recommended"
 - "Phased Success Plan and Adoption Journey"
 - "Outcomes, Objectives & Activities - Bus/Technical Milestones (3/6/12 month)"
 - "Architecture Notes:Producer (Deployment, Configuration, Integration); Consumer (Onboarding, Management,Monitoring)"

1. Mutual Customer Success Plan handoff to CSM for over 300k / Or AE and SA maintain after close

- The CSM would take the Success Plan started by the SA and add or update the following:
 - "Outcomes, Objectives & Activities - Bus/Technical Milestones (3/6/12 month)"
 - "Enablement Plan"
- The CSM for Renewal opportunities with ARR basis over 300k may also start a Mutual CSP from "13. Outcomes, Objectives & Activities - Bus/Technical Milestones (3/6/12 month)".
- In those cases, CSM is the one responsible to update the Customer Success Plan URL in the Renewal opportunity in Sales Force.
- The SA may optionally create a Customer Success Plan after the fact to add all the remaining context, to understand why was GitLab bought previously and what was the customer's expectation of value from GitLab

- In addition the CSM, when conducting cadences with the Customer, is expected record the following in the Mutual Customer Success Plan:

Content Element	Mutual Success Plan	Kick Off Call / Onboarding EBR
--	--	--
1.	Utilization per Features / Capabilities	Update Utilization upon Cadence
2.	Use Case Adoption in Prod	Update Use Case Adoption upon Cadence
3.	Support Tickets / Enhancements / issues	Update Tickets / Issues upon Cadence
4.	OnGoing / Planned Initiatives w/ Status	Update Milestones upon Cadence
5.	Outcome / Metric Updates	Update Metrics / Customer upon Cadence
6.	PS SOWs and RunBook	Attached

The CSM and the Customer Success Plan

[Click here to learn](#)